

Semaphore

When a process wants to access CS, check the semaphore variable. If the variable is set to 0, then sleep, Otherwise reduce its value by 1, and then continue to CS.

```
process A ( ) {  
    Semaphore_wait (&sem_variable);  
  
    CS ();  
  
    Semaphore_signal (&sem_variable);  
}
```

```
Semaphore_wait (Semaphore *sem) {  
    if (*sem == 0) sleep();  
    else  
        *sem = *sem - 1;  
}
```

```
Semaphore_signal (Semaphore *sem) {  
    if (*sem == 0) {  
        Find process P that is waiting  
        send_signal (&sem);  
    }  
}
```

```

    }
    *Sem + = 1;

```

```

}

```

1) Test-and-Set Lock → Usually used via mutex variables from C language.

```

pthread_mutex_t m = {0};

```

```

pthread_lock(&m);

```

CS(); // could be incrementing a global variable

```

pthread_unlock(&m);

```

Has higher overhead because each thread has to periodically check whether the lock is available or not.

2) Semaphore: → semaphore.h file

```

Semaphore *sem;

```

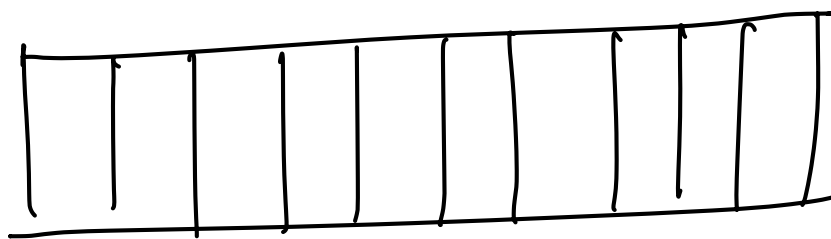
There are named Semaphore and unnamed Semaphores.

Synchronization across
any processes

only among
related processes.

Process Synchronization Problems

- 1) Producer-Consumer Problem: $\rightarrow$ You have a global queue where one process called producer wants to write characters. The other process called consumer wants to read characters written by the producer. You need to ensure that
- (i) the producer does not write to a full queue,
 - (ii) the consumer does not read from an empty queue, and
 - (iii) the consumer reads the earliest character written by the producer.



~~of~~ L
length

int a[5]; a[6] = 2;

```

producer() {
    if (num_chars == L) {
        wait(); [convert this to sem_wait]
        write(); num_chars = (num_chars + 1) % L;
    }
}

```

Two semaphores
 → one each
 for the num_chars
 and the queue =

```

consumer() {
    if (num_chars == 0) {
        wait();
        read();
        num_chars -= 1;
    }
}

```

```

while (num_chars == L);

```

Race condition can be avoided by using two
 semaphore variables

The challenge is to keep track of how each function
 has used its own semaphores.

Is there an alternative technique that is present?

At a language level, it is possible to use a lock to allow access to a single process to a part of a code.

Synchronized {

}

All ~~the~~ execution of the synchronized block is atomic in nature. It comes with heavy performance penalty, but is easy to use.

Global Interpreter Lock (GIL).

→ No thread can actually run in parallel.

Allows only sequential execution.

— Performance-intensive code depending on multicore systems cannot be done in Python.

— Multi-core parallelism requires creation of processes

and copying of address space.

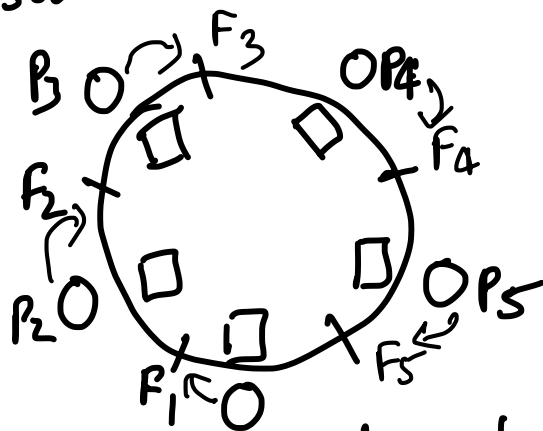
1) Task-level parallelism

2) Data-level parallelism $\rightarrow$ GPU's are most optimized for this.

.GPU's schedulers are different, provided by the drivers separately in PC's.

Dining Philosopher's Problem

Consider a place with a round table where five people sit. Each person (philosopher) needs



to use two forks P_i kept close to them when they try to eat.

```
person-action ( ) {  
    think();  
    grab_one_fork();  
    grab_second_fork();  
    eat();  
}
```

```
release_one_fork();  
release_second_fork();
```

```
}
```

This can easily lead to deadlock.

What is the solution?

→ Enforce a rule that a philosopher can only access a fork with a lower number first.

Deadlock is prevented by enforcement of strict sequencing of shared resource requests.

Whether to use them or not is a design choice. Linux does not enforce deadlock prevention.